Identificação de Incêndios Florestais utilizando Deep Learning

*Note: Sub-titles are not captured in Xplore and should not be used

1st José Janailson de Arruda Cunha
Centro de Informática
UFPE
Recife, Brasil
jjac@cin.ufpe.br

I. Objetivo

O objetivo deste projeto é desenvolver e avaliar modelos de redes neurais convolucionais (CNN) para realizar a classificação de imagens em duas categorias: com fogo e sem fogo. O objetivo é criar um sistema eficiente que possa identificar a presença de fogo em imagens, auxiliando em aplicações de monitoramento e detecção precoce de incêndios.

II. Justificativa

A detecção precoce de incêndios é crucial para mitigar os danos causados por esses eventos, especialmente em áreas florestais, industriais ou residenciais. O uso de técnicas de visão computacional baseadas em redes neurais convolucionais pode oferecer uma solução automatizada e de alta precisão para identificar a presença de fogo em imagens. Isso pode ser integrado em sistemas de monitoramento para alertar rapidamente autoridades ou sistemas de combate a incêndios, minimizando o impacto ambiental e econômico.

O desenvolvimento deste projeto justifica-se pela necessidade crescente de soluções tecnológicas que possam detectar incêndios de maneira eficiente e em tempo real. Além disso, o uso de CNNs permite explorar características visuais complexas nas imagens, aumentando a precisão da classificação quando comparado a métodos tradicionais.

III. Base de dados

O dataset utilizado é o "Forest Fire BigData" disponível no Kaggle. Este conjunto de dados contém imagens categorizadas em duas classes principais: imagens com fogo e imagens sem fogo. As imagens foram coletadas para treinar modelos de aprendizado profundo com o objetivo de realizar a detecção visual de incêndios em diferentes cenários.

IV. Análise exploratória dos dados

1. Distribuição das Classes: A base de dados contém 1475 imagens no conjunto de treino e 367 imagens no conjunto de validação. Essas imagens estão divididas de forma balanceada em duas classes: imagens contendo fogo e imagens sem fogo. Essa distribuição balanceada é crucial para evitar que o modelo se torne enviesado para uma das classes, garantindo um aprendizado mais robusto e generalizável.

2. Características Visuais das Imagens: As imagens possuem dimensões originais variáveis, mas foram redimensionadas para 128x128 pixels para padronização e otimização do tempo de treinamento. Todas as imagens foram normalizadas (escala de 0 a 1), o que é uma prática comum em aprendizado profundo para melhorar a estabilidade numérica e o desempenho dos modelos.

3. Pré-processamento: Normalização: Todas as imagens foram normalizadas através do ImageDataGenerator com rescale=1./255, o que ajuda a melhorar a eficiência do treinamento dos modelos, mantendo os valores de pixels em um intervalo padrão. Divisão em Treino e Validação: Foi utilizado o mesmo ImageDataGenerator com validation split=0.2 para separar os dados de forma automática em 804. Observação da Variabilidade: As imagens coletadas para esse projeto incluem cenários variados, o que é crucial para garantir que o modelo seja capaz de identificar o fogo em diferentes contextos, intensidades e condições de iluminação. Essa variabilidade é um desafio significativo para a detecção visual, uma vez que o fogo pode aparecer de diferentes formas em termos de cor, tamanho e intensidade.

5. Número de Imagens: A quantidade total de 1842 imagens foi suficiente para iniciar o treinamento de redes neurais, porém o aumento desse volume poderia levar a uma maior generalização e melhoria na acurácia do modelo. Além disso, técnicas de aumento de dados poderiam ter sido aplicadas para simular condições mais diversas e aumentar a robustez do modelo.

V. Experimentos

Os experimentos foram conduzidos com o objetivo de avaliar diferentes arquiteturas de redes neurais aplicadas à classificação de imagens contendo fogo ou sem fogo. A seguir estão os detalhes sobre cada modelo utilizado, suas

Identify applicable funding agency here. If none, delete this.

características, o processo de treinamento, e os resultados obtidos.

1. CNN Simples A CNN (Rede Neural Convolucional) foi projetada com uma arquitetura básica de duas camadas convolucionais seguidas por camadas de pooling, e uma camada totalmente conectada antes da saída final. Esse modelo foi construído do zero para atuar como baseline nos experimentos.

Arquitetura:

Duas camadas convolucionais:
Primeira: 32 filtros de tamanho 3x3 e ativação ReLU.
Segunda: 64 filtros de tamanho 3x3 e ativação ReLU.
Camadas de MaxPooling 2x2 após cada camada convolucional, para reduzir a dimensionalidade.
Camada Flatten para achatar a saída da última camada convolucional.
Camada totalmente conectada com 128 neurônios e ativação ReLU.
Dropout de 50 porcento para reduzir overfitting.
Camada de saída com ativação sigmoide para classificação binária (fogo ou sem fogo).

2. Transfer Learning: VGG16 A arquitetura VGG16 é um dos modelos mais populares para classificação de imagens e foi utilizada com pesos pré-treinados no ImageNet. A camada de classificação original foi removida, e novas camadas densas foram adicionadas para adaptar o modelo ao problema de detecção de incêndios.

Arquitetura:

Base VGG16 com suas 13 camadas convolucionais e pooling.
Camadas de classificação personalizadas:
Flatten para converter as saídas convolucionais em um vetor.
Camada densa com 128 neurônios e ativação ReLU.
Dropout de 50 porcento para regularização.
Camada de saída com ativação sigmoide para classificação binária.
Os pesos da base VGG16 foram congelados (não atualizados durante o treinamento), e apenas as camadas finais foram treinadas.

3. Transfer Learning: ResNet50 A arquitetura ResNet50 é mais moderna, introduzindo ”skip connections” ou ”residual connections”, o que facilita o treinamento de redes muito profundas sem perder desempenho devido ao problema de degradação do gradiente.

Arquitetura:
Base ResNet50 com 50 camadas profundas e conexões residuais.
Camadas de classificação personalizadas:
Flatten para converter as saídas convolucionais em um vetor.
Camada densa com 128 neurônios e ativação ReLU.
Dropout de 50 porcento para evitar overfitting.
Camada de saída com ativação sigmoide.
Assim como no VGG16, a base do ResNet50 foi congelada e apenas as camadas finais foram ajustadas durante o treinamento.

4. Transfer Learning: MobileNet O MobileNet é uma arquitetura otimizada para dispositivos móveis, projetada para ser leve e eficiente, sem sacrificar muito a precisão. Ele utiliza convoluções separáveis em profundidade, o que reduz drasticamente o número de parâmetros.

Arquitetura:
Base MobileNet com convoluções separáveis.
Camadas de classificação personalizadas:
Flatten para converter as saídas convolucionais em um vetor.
Camada densa com 128 neurônios e ativação ReLU.
Dropout de 50 porcento para regularização.
Camada de saída com ativação sigmoide.
A base MobileNet foi congelada, e apenas as camadas finais foram treinadas. line

VI. Análise dos resultados

A CNN (Rede Neural Convolucional)
Resultados:
Acurácia de Treino: 97.32 porcento
Acurácia de Validação: 92.10 porcento
AUC: 0.495
Análise: A CNN simples foi capaz de aprender bem no conjunto de treino, mas apresentou uma ligeira diferença de desempenho no conjunto de validação, o que indica um leve overfitting. O valor do AUC de 0.495 sugere que o modelo teve dificuldades em discriminar corretamente entre as classes ”fogo” e ”sem fogo”, apesar de sua acurácia razoável.

2. Transfer Learning: VGG16
Resultados:
Acurácia de Treino: 98.92 porcento
Acurácia de Validação: 94.28 porcento
AUC: 0.485
Análise: O VGG16 mostrou um excelente desempenho, com uma acurácia muito alta tanto no treino quanto na validação. A arquitetura pré-treinada foi capaz de transferir bem seu conhecimento para a nova tarefa. No entanto, o valor do AUC relativamente baixo indica que o modelo poderia estar aprendendo padrões excessivamente específicos dos dados de treino, o que pode ter prejudicado sua capacidade de discriminar de forma mais eficiente entre as duas classes.

3. Transfer Learning: ResNet50
Resultados:
Acurácia de Treino: 87.58 porcento
Acurácia de Validação: 90.46 porcento
AUC: 0.583
Análise: O ResNet50 teve um desempenho menor em relação aos outros modelos, tanto no treino quanto na validação. A acurácia foi menor, mas o AUC de 0.583 indicou que o modelo conseguiu discriminar melhor entre as classes em comparação com os demais modelos. No entanto, ainda houve espaço para melhorias, pois o modelo aparentava estar subajustado ao problema (menor acurácia de treino).

4. Transfer Learning: MobileNet
Resultados:

Acurácia de Treino: 99.48 porcento
Acurácia de Validação: 97.82 porcento
AUC: 0.506
Análise: O MobileNet apresentou os melhores resultados em termos de acurácia de validação (97.82 porcento), sendo o modelo mais eficiente para essa tarefa. Seu alto desempenho confirma a eficácia do MobileNet em transfer learning para tarefas de classificação de imagens, mesmo com um número relativamente pequeno de dados de treino. Contudo, o AUC de 0.506 sugere que, assim como os outros modelos, ele pode estar aprendendo padrões que não são necessariamente os mais discriminantes entre as duas classes.

VII. Conclusões e discussões

Os experimentos realizados com diferentes modelos de rede neural para a classificação de imagens de fogo e sem fogo mostraram que o MobileNet apresentou a melhor acurácia de validação (97.82 porcento) e uma acurácia de treino de 99.48 porcento, destacando-se pela eficiência em transfer learning. O VGG16 obteve uma acurácia de validação de 94.28 porcento e uma acurácia de treino de 98.92 porcento, mas seu AUC de 0.485 indicou dificuldades na discriminação entre classes. O ResNet50, apesar de ter a menor acurácia de treino (87.58 porcento), mostrou um desempenho melhor em AUC (0.583), sugerindo uma discriminação mais eficaz, enquanto a CNN Simples teve resultados intermediários, com acurácia de validação de 92.10 porcento e AUC de 0.495. No geral, todos os modelos apresentaram espaço para melhorias em termos de discriminação, mas o MobileNet se destacou como a opção mais promissora para esta tarefa específica.

References

[1] Dive into Deep Learning
[2] https://www.kaggle.com/datasets/mohammedborhanuddin/forest-fire-bigdata
[3] TensorFlow Documentation.
[4] Chollet, F. (2015). Keras: The Python Deep Learning library.
[5] LeCun, Y., et al. (2015). ”Deep learning.” Nature, 521(7553), 436-444.